the mutex and resumes waiting. This is why you need the `std::unique_lock` rather than the `std::lock_guard`—the waiting thread must unlock the mutex while it's waiting and lock it again afterward, and `std::lock_guard` doesn't provide that flexibility. If the mutex remained locked while the thread was sleeping, the data-preparation thread wouldn't be able to lock the mutex to add an item to the queue, and the waiting thread would never be able to see its condition satisfied.

Listing 4.1 uses a simple lambda function for the wait ❺, which checks to see if the queue is not empty, but any function or callable object could be passed. If you already have a function to check the condition (perhaps because it's more complicated than a simple test like this), then this function can be passed in directly; there's no need to wrap it in a lambda. During a call to `wait()`, a condition variable may check the supplied condition any number of times; however, it always does so with the mutex locked and will return immediately if (and only if) the function provided to test the condition returns `true`. When the waiting thread reacquires the mutex and checks the condition, if it isn't in direct response to a notification from another thread, it's called a *spurious wake*. Because the number and frequency of any such spurious wakes are by definition indeterminate, it isn't advisable to use a function with side effects for the condition check. If you do so, you must be prepared for the side effects to occur multiple times.

The flexibility to unlock a `std::unique_lock` isn't just used for the call to `wait()`; it's also used once you have the data to process but before processing it ❻. Processing data can potentially be a time-consuming operation, and as you saw in chapter 3, it's a bad idea to hold a lock on a mutex for longer than necessary.

Using a queue to transfer data between threads as in listing 4.1 is a common scenario. Done well, the synchronization can be limited to the queue itself, which greatly reduces the possible number of synchronization problems and race conditions. In view of this, let's now work on extracting a generic thread-safe queue from listing 4.1.

4.1.2. Building a thread-safe queue with condition variables

If you're going to be designing a generic queue, it's worth spending a few minutes thinking about the operations that are likely to be required, as you did with the thread-safe stack back in section 3.2.3. Let's look at the C++ Standard Library for inspiration, in the form of the `std::queue<>` container adaptor shown in the following listing.

Listing 4.2. `std::queue` interface

```
template <class T, class Container = std::deque<T> >
class queue {
public:
    explicit queue(const Container&);
    explicit queue(Container&& = Container());

    template <class Alloc> explicit queue(const Alloc&);
    template <class Alloc> queue(const Container&, const Alloc&);
    template <class Alloc> queue(Container&&, const Alloc&);
    template <class Alloc> queue(queue&&, const Alloc&);

    void swap(queue& q);

    bool empty() const;
```